

Confusion Matrices

See which mistakes a classifier makes—not only how many

Model Evaluation · Foundations of Machine Learning



The Binary Confusion Matrix

Diagonal — Correct predictions: TN and TP.

Off-diagonal — Errors: FP and FN.

Axis convention — This course: rows = actual, columns = predicted (sklearn's default). Libraries disagree — always read the labels.

	Pred: Negative	Pred: Positive
Actual: Negative	TN	FP
Actual: Positive	FN	TP

Same Accuracy, Different Failure

Model A — false-positive heavy

	Pred N	Pred P
Act N	750	150
Act P	30	70

Accuracy = $820/1000 = 0.82$. Many false alarms (150 FP), few misses (30 FN).

Model B — false-negative heavy

	Pred N	Pred P
Act N	870	30
Act P	50	50

Accuracy = $920/1000 = 0.92$. Few false alarms, many misses (50 FN, recall only 50%).

Accuracy alone cannot distinguish these two error profiles — you must look at the matrix to see which type of mistake a model makes.

Compute and Display the Matrix

Raw counts

`confusion_matrix` returns the table as a NumPy array.

Readable diagnostic

`ConfusionMatrixDisplay` adds axis labels and color intensity.

```
from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay,
)

y_pred = model.predict(X_test)          # never predict(X_train)
cm = confusion_matrix(y_test, y_pred)   # rows=actual, cols=predicted
print(cm)
ConfusionMatrixDisplay(
    cm, display_labels=["Negative", "Positive"],
).plot()
```

Multiclass: A $K \times K$ Error Map

Diagonal — Correct predictions for each of the K classes.

Off-diagonal structure — Which specific class pairs the model confuses, e.g. digit 4 $\leftrightarrow$ 9.

Per-class metrics — Treat class k as "positive," every other class as "negative."

$$P_k = \frac{C_{kk}}{\sum_i C_{ik}}, \quad R_k = \frac{C_{kk}}{\sum_j C_{kj}}$$

	P: 0	P: 4	P: 9
A: 0	50	0	0
A: 4	0	40	8
A: 9	0	6	44

Digit classifier, 148 test images. 0s are never confused; 4s and 9s — visually similar handwritten shapes — leak into each other in both directions.

Macro, Weighted, and Micro Averaging

Macro

Unweighted mean of per-class scores; every class counts equally regardless of size.

Weighted

Mean of per-class scores weighted by support (class size).

Micro

Pool every TP/FP/FN across all classes first, then compute one ratio; equals accuracy here.

Rule of thumb

Always check per-class recall when a rare class matters.

Support 920 / 970 / 10, rare class recall = 0.10 → **macro recall = 0.682**, but **weighted recall = 0.969** and **micro recall = 0.969**. The averages that weight by class size never show you the rare class is failing.

One Table, Every Classification Metric

Diagnose

Read error direction from off-diagonal cells.

Generalize

Use $K \times K$ for multiclass; per-class P/R from rows/columns.

Summarize carefully

Choose macro, weighted, or micro explicitly — do not accept a library default blindly.

Next: decide whether two measured model scores are genuinely different.